



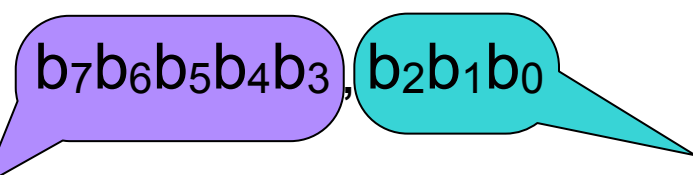
Universidade de Brasília

Departamento de Ciência da Computação

Aritmética Fracionária



Ponto Fixo

- Supor que o número inteiro é dividido em duas partes
 - parte inteira
 - parte fracionária
- A parte fracionária abrange os bits menos significativos e tem a seguinte representação:
 - $Q_i \Rightarrow i$ bits à direita do número constituem a parte fracionária
 - Ex: Q_3 : 
parte inteira parte fracionária



Representação de casas decimais em ponto fixo, complemento de 2

Ex.: 8 bits

Q3: $2^4 2^3 2^2 2^1 2^0, 2^{-1} 2^{-2} 2^{-3}$

Menor valor: 10000000 $-2^4 = -16$

Maior valor: 01111111 $2^3 + 2^2 + 2^1 + 2^0 + 2^{-1} + 2^{-2} + 2^{-3} = 15.875$

Q1: $2^6 2^5 2^4 2^3 2^2 2^1 2^0, 2^{-1}$

Menor valor: 10000000 $-2^6 = -64$

Maior valor: 01111111 $2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 + 2^{-1} = 63.5$

Q7: $2^0, 2^{-1} 2^{-2} 2^{-3} 2^{-4} 2^{-5} 2^{-6} 2^{-7}$

Menor valor: 10000000 $-2^0 = -1$

Maior valor: 01111111 $2^{-1} + 2^{-2} + 2^{-3} + 2^{-4} + 2^{-5} + 2^{-6} + 2^{-7} = 0.9921875$



■ Exemplo:

Considere 8 bits, calcule a representação em Q7

☐ $0.75 =$

☐ $-0.75 =$

☐ $0.3 =$

Truncamento? Arredondamento?

Considerando 16 bits, calcule a representação em Q15

☐ $0.75 =$

☐ $-0.75 =$

☐ $0.3 =$



Ponto Fixo

- Operações Matemáticas: da mesma forma que inteiros usando mesmo Q.

- ☐ Soma
- ☐ Subtração
- ☐ Multiplicação
- ☐ Divisão (ex.: 5/8)

Ex.: 8 bits

	Q0	Q2	Q7
01010001	81	20.25	0.6328125
+ 10111001	-71	-17.75	-0.5546875
00001010	10	2.50	0.0781250
00000110	06	1.5	0.046875
x 00001010	x10	x2.5	x0.078125
00000000 00111100	60	3.75	0.003662109375



Ponto Fixo

■ Problemas:

- ☐ Pequena Faixa Dinâmica
- ☐ Precisão depende da faixa dinâmica

■ Vantagens:

- ☐ Aritmética é simples e rápida
(Processador menor, mais rápido e mais barato)



Ponto Flutuante

- Representação de números muito grandes e muito pequenos

- 0,00000000000000001182721226716

- 16728387635120000000000000000000

- Notação Científica

- -2.34×10^{56}

normalizado

- $+0.0043 \times 10^{-7}$

- $+567.04 \times 10^3$

não - normalizado

- Tipos float e double em C

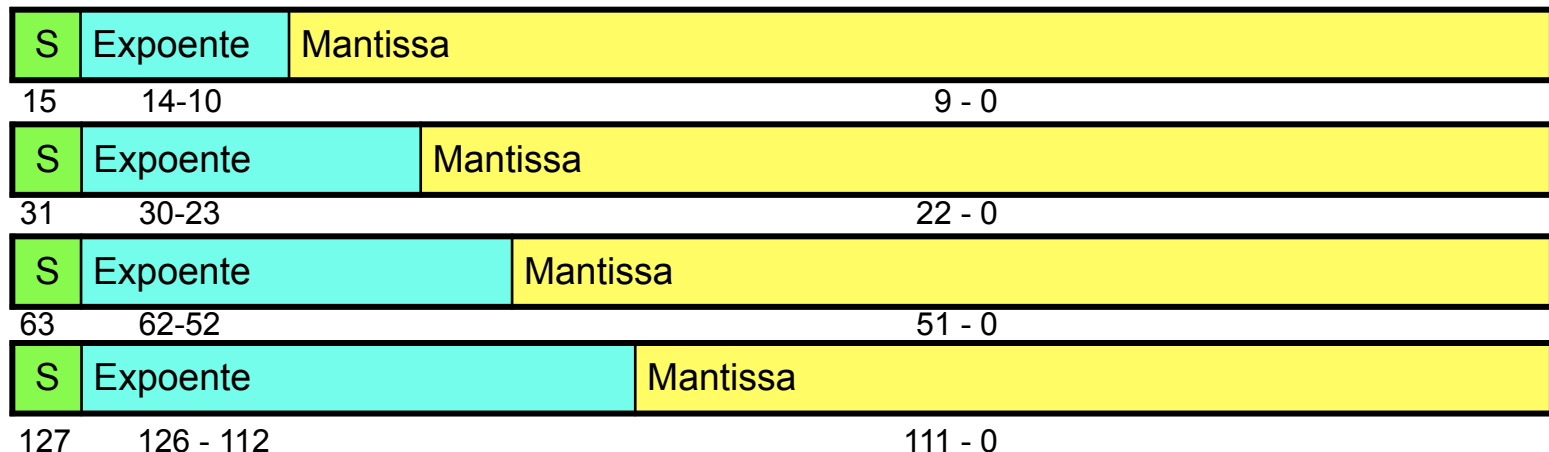


Ponto Flutuante

■ Padrão de ponto flutuante IEEE 754

- meia precisão: 16 bits => sinal: 1, exp: 5, mantissa: 10
- precisão simples: 32 bits => sinal: 1, exp: 8, mantissa: 23
- precisão dupla: 64 bits => sinal: 1, exp: 11, mantissa: 52
- precisão quádrupla: 128 bits => sinal: 1, exp: 15, mantissa: 112

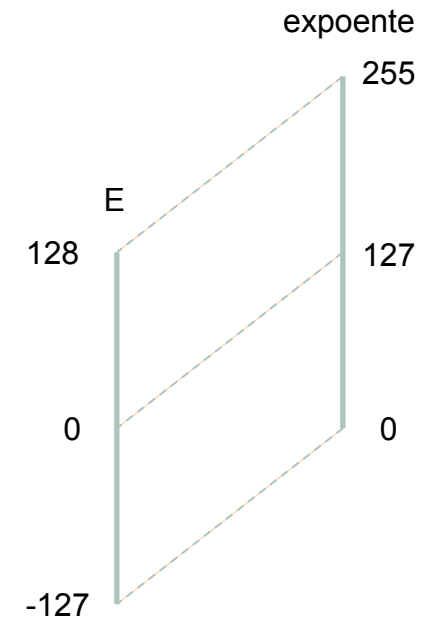
$$(-1)^{\text{sinal}} \times (1 + \text{fração}) \times 2^{(\text{expoente} - \text{bias})}$$



Padrão IEEE 754

■ Normalização

- $(\text{sinal})1.\text{bbbbbb} \times 2^e$
- Sinal: 1 $\Rightarrow$ negativo, 0 $\Rightarrow$ positivo
- Sempre um bit em 1 à esquerda da vírgula
- Expoente com deslocamento ($\text{exp} + \text{bias}$)
 - Bias = $2^{n-1} - 1$, onde n = número de bits do expoente
 - Precisão simples: $n = 8$, bias = 127
 - Expoente = $e - 127$
 - Precisão dupla: $n = 11$, bias = 1023
 - Expoente = $e - 1023$





Exemplo Padrão IEEE 754

■ Exemplo

- Representação de: -0.75_{10}
- Binário não normalizado: -0.11
- Normalizado: -1.1×2^{-1}
- Expoente = $127 - 1 = 126 = 01111110_2$
- Mantissa = $1000...00_2$
- Sinal: 1 (negativo)
- Precisão simples: $10111111010000000000...000$



Exemplo Padrão IEEE 754

- Dado o número em FP IEEE754: 0xC1100000 qual o número decimal representado?

1100 0001 0001 0000 0000 0000 0000 0000

1 10000010 0010000000000000000000000000

expoente = $130 - 127 = 3$

Mantissa: 1.001

Logo: $(-1)^1 \times 1.001 \times 2^3 = - (1001.0) = -9$



Precisão do Ponto Flutuante

■ Precisão relativa

- Menor diferença entre números consecutivos
- Simples: aproximadamente 2^{-23}
 - Equivale a $23 \times \log_{10} 2 \approx 23 \times 0.3 \approx 6$ casas decimais
- Dupla: aproximadamente 2^{-52}
 - Equivale a $52 \times \log_{10} 2 \approx 52 \times 0.3 \approx 16$ casas decimais



Limites na Representação

■ Overflow:

- ☐ número é muito grande para ser representado
- ☐ expoente positivo - ultrapassa o limite de representação

■ Underflow:

- ☐ número é muito pequeno para ser representado
- ☐ expoente negativo - ultrapassa o limite de representação



FP IEEE 754

- Como representar 0?

Precisão Simples		Precisão Dupla		Objeto
Expoente	Fração	Expoente	Fração	
0	0	0	0	0
0	$\neq 0$	0	$\neq 0$	$\pm$ Número desnormalizado
1-254	$\forall$	1-2046	$\forall$	$\pm$ Número Ponto Flutuante
255	0	2047	0	$\pm\infty$
255	$\neq 0$	2047	$\neq 0$	NaN

Obs.: Número desnormalizado: Considera 0 inicial na mantissa

Qual a faixa dinâmica dos números representáveis em precisão simples e dupla sem *overflow* ou *underflow*?



Número Denormalizado

- Expoente = 000...0 $\Rightarrow$ bit default é 0

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$$

- Menor que números normalizados
 - Estende limite do underflow
- Denormalizado com mantissa = 0000...00
 - Representa ± 0.0



Adição em Ponto Flutuante

- Considerar números decimais de 4 dígitos
 - $9.999 \times 10^1 + 1.610 \times 10^{-1}$
- 1. Alinhar pontos decimais
 - Deslocar número com menor expoente
 - $9.999 \times 10^1 + 0.016 \times 10^1$
- 2. Somar as mantissas
 - $9.999 \times 10^1 + 0.016 \times 10^1 = 10.015 \times 10^1$
- 3. Normalizar resultado e verif. over/underflow
 - 1.0015×10^2
- 4. Arredondar e renormalizar se preciso
 - 1.002×10^2



Adição em Ponto Flutuante

- Considerar números binários de 4 dígitos
 - $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ (0.5 + -0.4375)
- 1. Alinhar pontos binários
 - Deslocar número com menor expoente
 - $1.000_2 \times 2^{-1} + -0.1110_2 \times 2^{-1}$
- 2. Somar as mantissas
 - $1.000_2 \times 2^{-1} + -0.1110_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$
- 3. Normalizar resultado e verific. over/underflow
 - 1.000×2^{-4}
- 4. Arredondar e renormalizar se preciso
 - $1.000 \times 2^{-4} = 0.0625$

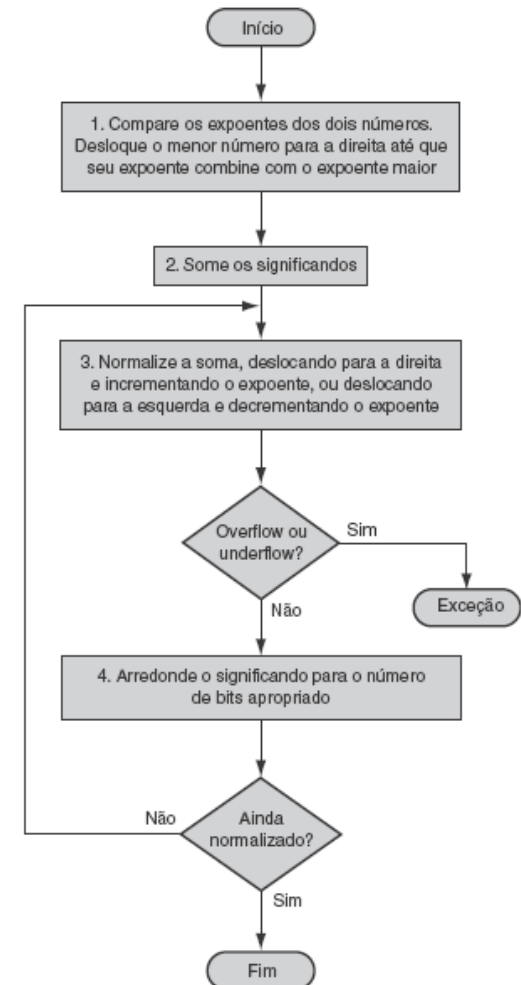
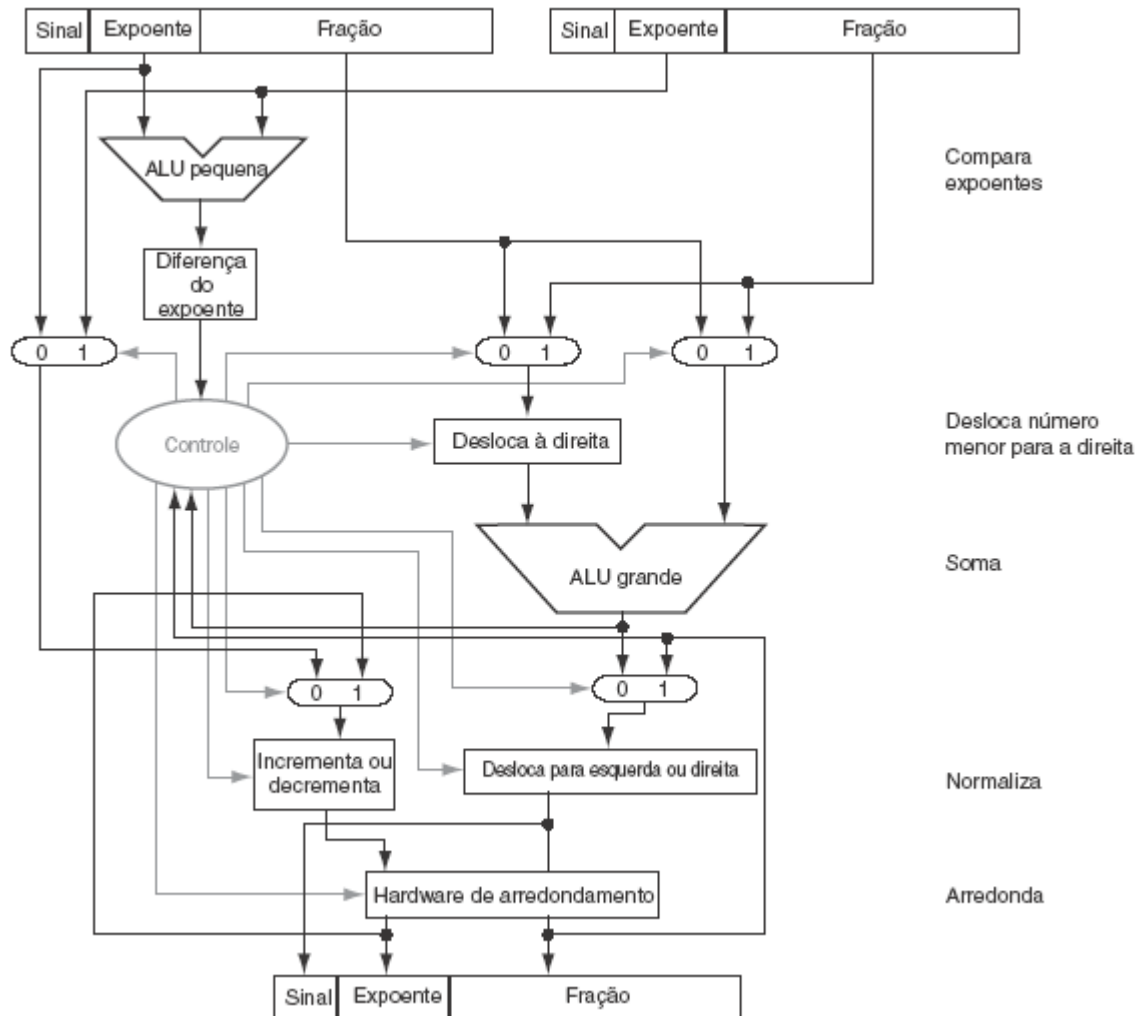


Adição em Ponto Flutuante - HW

- Muito mais complexa que adição de inteiros
- Executar todo o procedimento em um ciclo de relógio iria requerer um período muito grande
 - Relógio mais lento impactaria todo o processador
- Usualmente realizada em vários ciclos de relógio
 - Poderia executar em *pipeline* (cada



Adição em Ponto Flutuante - HW





Multiplicação em Ponto Flutuante

- Exemplo decimal, 4 dígitos
 - $1.110 \times 10^{10} \times 9.200 \times 10^{-5}$
- Somar expoentes (subtrair bias, se houver)
 - $10 + - 5 = 5$
- Multiplicar mantissas
 - $1.110 \times 9.200 = 10.212 \Rightarrow 10.212 \times 10^5$
- Normalizar resultado e tratar estouro
 - 1.0212×10^6
- Arredondar e renormalizar, se necessário
 - 1.021×10^6
- Determinar o sinal do resultado em função dos operandos



Multiplicação em Ponto Flutuante

- Exemplo binário, 4 dígitos
 - $1.000 \times 2^{-1} \times -1.110 \times 2^{-2}$
- Somar expoentes (subtrair bias, se houver)
 - Sem bias: $-1 + -2 = -3$
 - Com bias: $(-1 + 127) + (-2 + 127) = -3 + 254 - 127 = -3 + 127$
- Multiplicar mantissas
 - $1.000 \times 1.110 = 1.110 \Rightarrow 1.110 \times 2^{-3}$
- Normalizar resultado e tratar estouro
 - 1.110×2^{-3}
- Arredondar e renormalizar, se necessário
 - 1.110×2^{-3}
- Determinar o sinal do resultado em função dos operandos
 - -1.110×2^{-3}



Ponto Flutuante: Arredondamento

- O IEEE 754 permite 4 tipos de arredondamentos
 - Sempre para $+\infty$ (cima, *ceil*): $2.11 \Rightarrow 2.2$ $2.15 \Rightarrow 2.2$ $2.19 \Rightarrow 2.2$
 - Sempre para $-\infty$ (baixo, *floor*): $2.11 \Rightarrow 2.1$ $2.15 \Rightarrow 2.1$ $2.19 \Rightarrow 2.1$
 - Truncamento: Despreza os bits menos significativos (*trunc*)
 - $+1.01101 = 1.40625 \Rightarrow +1.011 = 1.375$
 - Ao mais próximo (*round*): $2.11 \Rightarrow 2.1$ $2.19 \Rightarrow 2.2$ $2.15 \Rightarrow ? \uparrow \downarrow ?$
- Estatisticamente coerente:
- Ao dígito par: $2.15 \Rightarrow 2.2$ $2.25 \Rightarrow 2.2$

Obs.: Em precisão limitada $(x+y)+z \neq x+(y+z)$

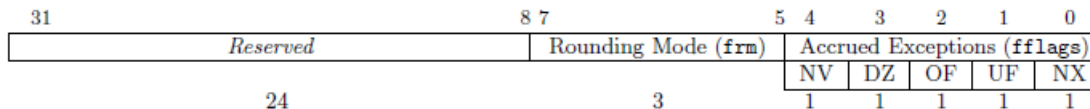
$$x + (y + z) = -1,5 \times 10^{38} + (1,5 \times 10^{38} + 1,0) = 0,0$$

$$(x + y) + z = (-1,5 \times 10^{38} + 1,5 \times 10^{38}) + 1,0 = 1,0$$



Ponto Flutuante: RV32IMFD

- F e D são extensões da ISA para ponto flutuante simples e duplo
- 32 Registradores de precisão simples/dupla: f0, f1,..., f30, f31
- Se F e D estão juntas, temos registradores de 64 bits. A extensão F utiliza os 32 bits iniciais
- Floating-point Control and Status Register (FCSR)



Modos de Arredondamento:

000: ao número par (round)

001: para o zero (round)

010: para baixo (floor)

011: para cima (ceil)

100: para a maior magnitude (round)

Flags de exceção

NV = Operação Inválida

DZ = Divisão por zero

OF = Overflow

UF = Underflow

NX = Inexato

Control and Status		
Registers		Floating Point
Name	Number	Value
ft0	0	0x0000000000000000
ft1	1	0x0000000000000000
ft2	2	0x0000000000000000
ft3	3	0x0000000000000000
ft4	4	0x0000000000000000
ft5	5	0x0000000000000000
ft6	6	0x0000000000000000
ft7	7	0x0000000000000000
fs0	8	0x0000000000000000
fs1	9	0x0000000000000000
fa0	10	0x0000000000000000
fa1	11	0x0000000000000000
fa2	12	0x0000000000000000
fa3	13	0x0000000000000000
fa4	14	0x0000000000000000
fa5	15	0x0000000000000000
fa6	16	0x0000000000000000
fa7	17	0x0000000000000000
fs2	18	0x0000000000000000
fs3	19	0x0000000000000000
fs4	20	0x0000000000000000
fs5	21	0x0000000000000000
fs6	22	0x0000000000000000
fs7	23	0x0000000000000000
fs8	24	0x0000000000000000
fs9	25	0x0000000000000000
fs10	26	0x0000000000000000
fs11	27	0x0000000000000000
ft8	28	0x0000000000000000
ft9	29	0x0000000000000000
ft10	30	0x0000000000000000
ft11	31	0x0000000000000000



Ponto Flutuante: RV32IMFD

■ Permite operações com precisão simples e dupla

- Adição: fadd.s e fadd.d # fadd.s f0, f1, f2
- Subtração: fsub.s e fsub.d # fsub.d f0, f2, f4
- Multiplicação: fmul.s e fmul.d # fmul.s f0, f1, f2
- Divisão: fdiv.s e fdiv.d # div.d f0, f4, f8
- Comparação:
 - feq.s t1, f1, f2 # t1 = (f1 == f2) ? 1 : 0
 - fle.s t1, f1, f2 # t1 = (f1 <= f2) ? 1 : 0
 - flt.s t1, f1, f2 # t1 = (f1 < f2) ? 1 : 0
 - fmax.s f0, f1, f2 # f0 = (f1 > f2) ? f1 : f2
 - fmin.s f0, f1, f2 # f0 = (f1 < f2) ? f1 : f2



Ponto Flutuante: RV32IMFD

□ Acesso à memória

- flw f0, 0(s0)

f0 = mem[s0 + 0]

- fsw f1, 4(s0)

mem[s0 + 4] = f1

□ Movimentação de dados

- mv.x.s t0, f0

t0 = f0

- mv.s.x f1, t1

f1 = t1

□ Conversão de dados

- fcvt.s.w f0, t0

f0 = (float)t0

- fcvt.s.wu f0, t0

f0 = (float)(unsigned)t0

- fcvt.w.s t0, f0

t0 = (int)f0

- fcvt.wu.s t0, f0

t0 = (unsigned)f0



Ex: Conversor de Temperatura

```
float fahr2c(float fahr)
{
    return 5.0*(fahr-32.0) / 9.0;
}
```

constantes no segmento de dados ou em imediatos (o que é + eficiente?)

.data

const5: .float 5.0

const9: .float 9.0

.text

fahr2c: la t0,const5

flw ft0,0(t0) # ft0=5.0

flw ft1,4(t0) # ft1=9.0

li t0,32

fcvt.s.w ft2,t0 # Não requer acesso à memória de dados!!!

fsub.s fa0,fa0,ft2

fmul.s fa0,fa0,ft0

fdiv.s fa0,fa0,ft1

ret



Complexidades do ponto flutuante

- As operações aritméticas são mais complexas
- Além do *overflow* podemos ter *underflow*
- A precisão pode ser um grande problema
 - O IEEE 754 mantém dois bits extras, guarda e arredondamento
 - Vários modos de arredondamento
 - Permite ajustar o comportamento numérico
- Nem todas unidades de FP implementam tudo
 - A maioria das linguagens e bibliotecas de FP usam apenas opções padrão



Observações

- A aritmética de computador é restrita por uma precisão limitada
- Os padrões de bit não têm um significado inerente mas existem padrões
 - complemento de dois
 - ponto flutuante IEEE 754
- As instruções de computador determinam o “significado” dos padrões de bit
- O desempenho e a precisão são importantes; portanto, existem muitas complexidades nas máquinas reais
- A escolha do algoritmo é importante e pode levar a otimizações de hardware para espaço e tempo (por exemplo, multiplicação)